Comparison of the Efficiency of Two Methods of Finding the Fourth Largest Element in an Unsorted Sequence

Technical report

Rysbek Dokturbaev, student no. 35691

IT Engineering, Group 01

29 October 2026

1. Introduction and goal

The task is to find the fourth largest element of an unsorted sequence of numbers. Two classical methods are compared: **FourPass** (method 3.1), which scans the whole sequence four times and finds the largest, second largest, third largest and finally the fourth largest element in successive passes, and **SinglePass** (method 3.2), which scans the sequence once and keeps the four largest values seen so far. The goal of this report is not to prove which method is better on paper but to compare their *actual* running times on a real computer, for large data sets, using repeated measurements, and to explain the observed results.

Both methods have linear time complexity O(N): FourPass performs about 4N element visits and SinglePass N element visits, and both need only a constant amount of extra memory (apart from the array itself). The interesting question is how the difference between 4N and N visits translates into measured time.

2. The two methods

2.1 FourPass

In pass j (j = 1..4) the algorithm scans the elements that have not yet been selected, finds the largest of them and swaps it to position j. After the fourth pass the element at position 4 is the fourth largest. Duplicates are counted as separate elements (for the sequence 9, 9, 9, 9, 1 the fourth largest is 9). The method reorders the input array, so in the experiment it always runs *after* SinglePass on the same data.

2.2 SinglePass

The algorithm keeps four variables first >= second >= third >= fourth. For each element it compares the element with first, second, third and fourth in that order and inserts it at the right position, shifting the smaller values down. The input array is not modified. For random data, a new element is rarely among the four largest, so in almost all iterations only the comparisons are executed and no values are shifted.

2.3 Correctness test

Before any measurement both methods were checked by a self-test on 1000 random arrays of length 4 to 199 with many duplicate values (values 0..49), comparing the result with the fourth element of the array sorted in descending order. In addition, in every experimental run the two methods were compared with each other and the program stops if the results differ. No difference was ever detected.

3. Experimental setup

3.1 Hardware and software

Item	Value
Computer	Laptop, Windows
CPU	Intel Core i7-13650HX (13th gen), 14 cores / 20 logical processors, base 2.60 GHz
CPU cache	L1 1.2 MB, L2 11.5 MB, L3 24.0 MB
RAM	15.6 GB usable (DDR5)
Language / runtime	C#, .NET SDK 10.0.400, Release configuration
Parallelism	none - both algorithms are single-threaded

3.2 Data and time measurement

The elements are 32-bit integers (`int`) generated with `Random.Next()`, i.e. uniformly distributed in $0..2^{31}-2$. An array of N integers occupies 4N bytes. The time of each algorithm is measured with `System.Diagnostics.Stopwatch`, started immediately before and stopped immediately after the call of the search method. Generating the data is **not** included in the measured time.

To avoid repeated allocation of gigabytes of memory, one array of the maximal size is allocated once per experiment, and for each run its first N elements are refilled with new random numbers. Thus each of the 10 runs at a measurement point uses a different random sequence, and in every run both methods work on exactly the same data.

3.3 Choosing the range of N

A calibration run was used to find the order of magnitude of N. Its results (one run per size) are in Table 1.

N	SinglePass [s]	FourPass [s]
10 000 000	0.007	0.016
20 000 000	0.014	0.030
40 000 000	0.028	0.061
80 000 000	0.052	0.150
160 000 000	0.111	0.245
320 000 000	0.335	0.627
640 000 000	0.478	1.271
1 280 000 000	2.098	4.184

Table 1. Calibration run (single measurement per size, earlier version of the program that allocated a new array for every size, so memory pressure was higher than in the final experiment).

The two requirements of the assignment and how they could be met on this computer:

- **Faster algorithm should take at least about 1 s (and certainly not less than 0.1 s).** SinglePass needs 0.62 s at the smallest measurement point and 1.10 s at the largest one. The 0.1 s minimum is therefore satisfied everywhere, but the 1 s target is reached only at the largest point (mean 1.10 s); at $N = 1.1 \times 10^9$ the mean is 0.96 s and at the lower points it is lower still.
- **Slower algorithm should take about a minute.** This was **not** reached. The size of the data set is limited by RAM: the largest array that fits safely is about 4.8 GB (1 200 000 000 elements) on a machine with 15.6 GB, of which a considerable part is used by the operating system and other processes. At that size FourPass runs only about 2.4 s. A linear extrapolation of the measured times

gives that about 22 x 10^9 elements (about 89 GB of data) would be needed for one minute, which is far beyond the memory of this computer. A larger N could only be reached with a smaller numeric type or with data stored outside RAM, which would change the nature of the experiment.

This is the reason for using a smaller data set for the slower algorithm than the assignment suggests; the whole experiment below was therefore done in the range that the memory of the computer allows.

3.4 Measurement points and procedure

The segment between the smallest and the largest N was divided into four equal parts, which gives five equidistant measurement points: 800 000 000, 900 000 000, 1 000 000 000, 1 100 000 000, 1 200 000 000 elements. At each point 10 experiments were performed; in each of them a new random sequence was generated and both methods were timed. For each point and each method the arithmetic mean and the sample standard deviation (with n-1 in the denominator) of the 10 times were computed, which gives 2 x 5 means and 2 x 5 standard deviations.

3.5 Measurement conditions

The experiment is sensitive to what else the computer is doing. An earlier attempt was run from inside the Visual Studio terminal with other programs open: the times of single runs varied strongly (for example FourPass took from about 1.7 s to 16 s in single runs) and the program finally ran out of memory. Those results were discarded. The final experiment was run from a plain PowerShell window with the development environment, the browser and other applications closed.

4. Results

Table 2 contains the main result of the experiment: the mean time and the standard deviation of 10 runs for each of the two methods at each of the five points.

N (elements)	SinglePass mean [s]	SinglePass std. dev. [s]	FourPass mean [s]	FourPass std. dev. [s]
800 000 000	0.616	0.129	1.451	0.230
900 000 000	0.703	0.112	1.604	0.203
1 000 000 000	0.842	0.148	1.713	0.254
1 100 000 000	0.959	0.217	2.383	0.399
1 200 000 000	1.103	0.181	2.430	0.421

Table 2. Mean running time and sample standard deviation of 10 runs per measurement point.

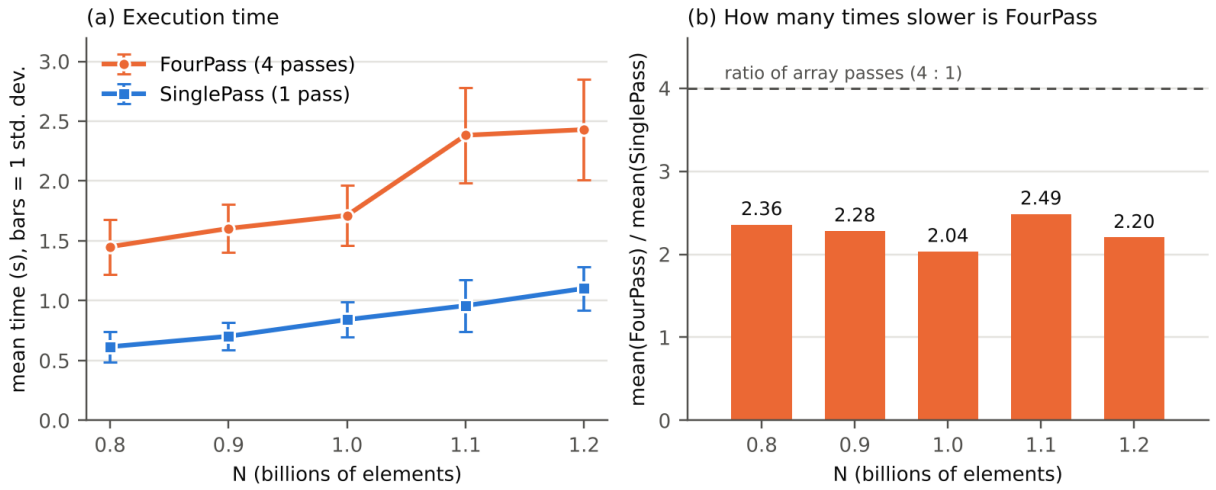


Figure 1. (a) Mean running times with one standard deviation as error bars. (b) Ratio of the mean times of the two methods; the dashed line marks the ratio of the number of passes over the array (4 : 1).

N (elements)	FourPass / SinglePass	SinglePass [ns / element]	FourPass [ns / element]	Rel. spread Single [%]	Rel. spread Four [%]
800 000 000	2.36	0.77	1.81	21	16
900 000 000	2.28	0.78	1.78	16	13
1 000 000 000	2.04	0.84	1.71	18	15
1 100 000 000	2.49	0.87	2.17	23	17
1 200 000 000	2.20	0.92	2.02	16	17

Table 3. Quantities derived from Table 2. Relative spread = standard deviation / mean.

5. Analysis

5.1 SinglePass is faster at every measurement point

At all five points SinglePass has the smaller mean time, and the ratio of the two means is between 2.04 and 2.49 (average 2.27). The difference between the means is at least 2.9 times larger than the combined standard deviation of the two methods at the same point, so the result is not an effect of the noise of single measurements.

5.2 Both methods are linear in N

Between the smallest and the largest N the number of elements grows by a factor of 1.50. In the same range the mean time of SinglePass grows by a factor of 1.79 and the mean time of FourPass by 1.67. A straight line fitted to the five means describes the data (R^2 = 0.994 for SinglePass and 0.895 for FourPass), with slopes of 1.23 ns and 2.74 ns per additional element, respectively. The fit is clearly better for SinglePass; the FourPass means are noisier (Section 5.4). Overall the data agree with the theoretical O(N) complexity of both methods.

The time per element is not perfectly constant: for SinglePass it increases from 0.77 ns to 0.92 ns. Since the neighbouring points differ by less than one standard deviation in several cases (for example SinglePass at 1.0×10^9 and 1.1×10^9), part of this increase may be noise; with five points and ten runs per point we cannot say that the growth is a real departure from linearity.

5.3 Why the ratio is about 2.3 and not 4

FourPass reads the whole array four times, so it might be expected to be four times slower. The measured ratio is about 2.3. Counting operations shows why the expectation was too simple. FourPass performs one comparison per element in each pass, that is about 4N comparisons in total. SinglePass compares every element (for random data, almost always) with all four stored values, which is also about 4N comparisons. The number of comparisons is thus similar, while the number of memory reads differs by a factor of four: at N = 1 200 000 000 FourPass reads about 19.2 GB from memory and SinglePass about 4.8 GB. A single pass of FourPass takes on average only 0.36 s at the smallest N, less than the whole SinglePass (0.62 s), which is consistent with SinglePass doing more work per element.

The arrays (up to 4.8 GB) are about 200 times larger than the 24 MB L3 cache, so in every pass the data come from main memory. The effective reading speed was however only about 4.4 GB/s for SinglePass and 7.9 GB/s for FourPass at the largest N, which is well below what the memory of such a computer can deliver. We did not profile the program, so we cannot separate how much of the time is spent waiting for memory and how much on executing the comparisons; the data only show that reducing the number of passes from four to one gives a speed-up of about 2.3, not 4.

5.4 Variability of the measurements

The standard deviation is between 16% and 23% of the mean for SinglePass and between 13% and 17% for FourPass. This is a large spread for a deterministic algorithm on data of the same size, and it shows that the measured time depends strongly on the state of the computer. Likely sources are other processes in Windows, changes of the CPU frequency and temperature in a laptop processor (the Task Manager showed 3.89 GHz during use, against a base of 2.60 GHz), and the high memory use of the system (9.1 of 15.6 GB was in use when the Task Manager screenshot was taken). The earlier, discarded attempt in which other programs were running (Section 3.5) had a much larger spread, which supports the explanation that the noise comes from the environment and not from the data. We did not test these causes separately.

The point $N = 1.1 \times 10^9$ shows a relatively large jump in the FourPass mean (from 1.71 s to 2.38 s), while the next point increases only to 2.43 s. With a standard deviation of about 0.40 s at that point, this is probably noise and should not be interpreted as a property of the algorithm.

6. Conclusions

- Both methods were implemented, tested for correctness and measured on five equidistant sizes between 800 000 000 and 1 200 000 000 elements, with 10 runs per point and a new random sequence in each run.
- The one-pass method (SinglePass) was faster at every point, by a factor of 2.0 to 2.5. In addition it does not modify the input array, which FourPass does.
- The running times of both methods are proportional to N, as expected for O(N) algorithms. The two methods have the same complexity class and differ in the constant factor.
- The speed-up is smaller than the factor of 4 suggested by the number of passes, because the number of comparisons of the two methods is similar and only the number of memory reads differs.
- The measurements have a considerable spread (13-23% of the mean), which shows how much results of a timing experiment on a laptop depend on conditions.

Limitations. All measurements come from one computer and one programming language. The range of N is narrow (a factor of 1.5) because of the memory limit, the slower algorithm did not reach the one-minute target of the assignment, only 10 runs were made per point, and we did not profile the programs, so the explanations in Sections 5.3 and 5.4 are plausible interpretations and not proven causes.

Appendix A. Source code (C#)

```
using System;
using System.Diagnostics;
using System.IO;
using System.Linq;

class Program
{
    static void Main(string[] args)
    {
        SelfTest();

        if (args.Length > 0 && args[0] == "calibrate") { Calibrate(); return; }

        int nMin = int.Parse(args[0]);
        int nMax = int.Parse(args[1]);
        RunExperiments(nMin, nMax);
    }

    public static int SinglePass(int[] array) => SinglePass(array, array.Length);
    public static int FourPass(int[] array) => FourPass(array, array.Length);

    // Method 3.2: one pass, keep the 4 largest values seen so far.
    public static int SinglePass(int[] array, int length)
    {
        if (array == null || length < 4)
            throw new ArgumentException("Array must contain at least 4 elements.");

        int first = int.MinValue, second = int.MinValue,
            third = int.MinValue, fourth = int.MinValue;

        for (int i = 0; i < length; i++)
        {
            int number = array[i];
            if (number > first)
            {
                fourth = third; third = second; second = first; first = number;
            }
            else if (number > second)
            {
                fourth = third; third = second; second = number;
            }
            else if (number > third)
            {
                fourth = third; third = number;
            }
            else if (number > fourth)
            {
                fourth = number;
            }
        }
        return fourth;
    }

    // Method 3.1: four passes; pass j moves the j-th largest element to position j.
    // NOTE: this method reorders the array.
    public static int FourPass(int[] array, int length)
    {
        if (array == null || length < 4)
            throw new ArgumentException("Array must contain at least 4 elements.");

        int result = 0;
        for (int j = 0; j < 4; j++)
        {
            int max = array[j];
            int index = j;
            for (int a = j + 1; a < length; a++)
            {
                if (array[a] > max) { max = array[a]; index = a; }
            }
            (array[j], array[index]) = (array[index], array[j]);
            result = array[j];
        }
        return result;
    }

    static void Fill(int[] data, int length, Random rnd)
    {
        for (int i = 0; i < length; i++) data[i] = rnd.Next();
    }

    static double TimeSingle(int[] data, int n, out int res)
    {
        var sw = Stopwatch.StartNew();
        res = SinglePass(data, n);
```

```
        sw.Stop();
        return sw.Elapsed.TotalSeconds;
    }

    static double TimeFour(int[] data, int n, out int res)
    {
        var sw = Stopwatch.StartNew();
        res = FourPass(data, n);
        sw.Stop();
        return sw.Elapsed.TotalSeconds;
    }

    // Correctness test against a full sort (small arrays, with duplicates).
    static void SelfTest()
    {
        var rnd = new Random(1);
        for (int t = 0; t < 1000; t++)
        {
            int n = rnd.Next(4, 200);
            var arr = Enumerable.Range(0, n).Select(_ => rnd.Next(0, 50)).ToArray();
            int expected = arr.OrderByDescending(x => x).ElementAt(3);
            if (SinglePass(arr) != expected || FourPass((int[])arr.Clone()) != expected)
                throw new Exception("Self-test FAILED");
        }
        Console.WriteLine("Self-test OK");
    }

    // Used to find the usable range of N on the given machine.
    static void Calibrate()
    {
        var rnd = new Random(42);
        var data = new int[1_400_000_000];
        for (int n = 100_000_000; n <= 1_400_000_000; n += 300_000_000)
        {
            Fill(data, n, rnd);
            double ts = TimeSingle(data, n, out int a);
            double tf = TimeFour(data, n, out int b);
            Console.WriteLine($"N={n,14:N0}  single={ts:F3}s  four={tf:F3}s  ok={a == b}");
        }
    }

    // 5 equidistant measurement points, 10 runs each, fresh random data every run.
    static void RunExperiments(int nMin, int nMax)
    {
        var rnd = new Random();
        var data = new int[nMax];              // one allocation for the whole experiment

        using var csv = new StreamWriter("results.csv");
        csv.WriteLine("N,run,single_s,four_s");
        var summary = new System.Text.StringBuilder();
        summary.AppendLine("N,single_mean,single_std,four_mean,four_std");

        for (int p = 0; p < 5; p++)
        {
            int n = nMin + (int)((long)(nMax - nMin) * p / 4);
            var ts = new double[10];
            var tf = new double[10];
            for (int r = 0; r < 10; r++)
            {
                Fill(data, n, rnd);                    // data generation is NOT timed
                ts[r] = TimeSingle(data, n, out int a);
                tf[r] = TimeFour(data, n, out int b);  // FourPass reorders data, so it runs second
                if (a != b) throw new Exception("Results differ!");
                csv.WriteLine($"{n},{r + 1},{ts[r]:F6},{tf[r]:F6}");
                csv.Flush();
                Console.WriteLine($"N={n:N0} run {r + 1}: single={ts[r]:F3}s four={tf[r]:F3}s");
            }
            summary.AppendLine($"{n},{ts.Average():F6},{Std(ts):F6},{tf.Average():F6},{Std(tf):F6}");
            File.WriteAllText("summary.csv", summary.ToString());
        }
        Console.WriteLine(summary);
    }

    // Sample standard deviation (n - 1).
    static double Std(double[] x)
    {
        double m = x.Average();
        return Math.Sqrt(x.Sum(v => (v - m) * (v - m)) / (x.Length - 1));
    }
}
```